(page 51)

3.8.2 TP Forme normale d'Hermite et transformations unitaires

Mettez en oeuvre l'algorithme décrit dans la section 3.4 pour le calcul d'une forme normale d'Hermite et la solution d'un système d'équations linéaires en nombres entiers. Dans la suite on propose des pistes pour la solution du problème.

1. Recherchez (et au besoin mettez en oeuvre) une description de l'algorithme étendu d'Euclide qui à partir de a et b, calcule x et y tels que $pgcd(a, b) = ax + by$.
2. A partir de la matrice A de dimension $m \times n$, construisez une matrice U de dimension $n \times n$ qui est initialement la matrice identité.

Pour transformer A en forme normale d'Hermite on multiplie A à droite par une série de matrices unimodulaires. Pratiquement, il s'agit à chaque étape de modifier 1 ou 2 colonnes de la matrice A. On effectue exactement les mêmes manipulations sur la matrice U et de cette façon à la fin du calcul de la forme normale d'Hermite H on aura aussi calculé la matrice unimodulaire U telle que $AU = H$. Les questions 3–6 qui suivent proposent une décomposition de cette tâche.

3. Programmez une fonction qui combine les colonnes i et j pour $i < j$ de façon à rendre l'élément $a_{i,j}$ nul. Cette fonction va utiliser l'algorithme d'Euclide étendu.
4. Programmez une fonction qui inverse le signe de la colonne i.
5. Programmez une fonction qui manipule la colonne j avec $j < i$ de façon à remplacer $a_{i,j}$ par le reste de la division de $a_{i,j}$ par $a_{i,i}$.
6. Programmez une fonction qui utilise les trois fonctions précédentes pour calculer une forme normale d'Hermite H et une matrice unimodulaire U telle que $AU = H$.
7. Pour un vecteur b donné, programmez une fonction qui va résoudre le système $Hy = b$ si possible et qui ensuite calcule les solutions Uy du système $Ax = b$. Testez sur les exemples 15 et 16.

(page 60)

3.8.9 TP Équilibre de Nash

On considère une *généralisation* des jeux à somme nulle décrits dans la section 2.6.2. Il s'agit toujours d'un jeu à *deux joueurs* qu'on appelle *Ligne* et *Colonne*, qui jouent un coup une seule fois et en même temps. Le jeu est décrit par deux matrices A et B de dimension $m \times n$ dont on dénote les coefficients avec $a_{i,j}$ et $b_{i,j}$, respectivement. *Ligne* dispose de m coups (on dit aussi stratégies pures) qui correspondent aux lignes des matrices et *Colonne* de n coups qui correspondent aux colonnes des matrices. Si *Ligne* joue le coup i et *Colonne* le coup j alors le gain de *Ligne* et de *Colonne* sont $a_{i,j}$ et $b_{i,j}$, respectivement. Dans les jeux à somme nulle, on a toujours $b_{i,j} = -a_{i,j}$ et ainsi une seule matrice suffit pour décrire le jeu.

Une *stratégie mixte* d'un joueur est une distribution de probabilités sur les coups du joueur. Si *Ligne* joue x et *Colonne* joue y alors les gains attendus (l'espérance) pour *Ligne* et *Colonne* sont $x^T Ay$ et $x^T By$, respectivement. Une stratégie mixte x de *Ligne* est une *meilleure réponse* à la stratégie mixte y de *Colonne* si pour toute autre stratégie mixte z de *Ligne* on a : $x^T Ay \geq z^T Ay$. Dans ce cas, *Ligne* n'a aucun intérêt à changer de stratégie tant que *Colonne* ne change pas la sienne. Dans le même esprit, une stratégie mixte y de *Colonne* est une meilleure réponse à la stratégie x de *Ligne* si pour toute autre stratégie z de *Colonne* on a : $x^T Ay \geq x^T Az$.

Un *équilibre* est un couple de stratégies mixtes (x, y) de *Ligne* et de *Colonne* telles que x est une meilleure réponse à y et y est une meilleure réponse à x. Dans sa thèse de doctorat, John Nash [Nas50] a montré que tout jeu (fini) admet un équilibre (de Nash). L'équilibre n'est pas forcement unique et on ne connaît pas d'algorithme polynomial pour en calculer un. Dans la suite, on se propose de formuler la recherche d'un équilibre comme un problème d'optimisation linéaire en nombres entiers.[6]

Équilibre de Nash et théorème minimax

On analyse d'abord la notion d'équilibre de Nash dans le cadre des jeux à somme nulle considérés dans la section 2.6.2.[7] On suppose que le jeu à somme nulle est décrit par une matrice A et qu'on a implicitement $B = -A$. Dans ce contexte, les joueurs *Ligne* et *Colonne* s'appellent aussi *Max* et *Min*, respectivement.

1. Soit $(\overline{x}, \overline{y})$ un équilibre de Nash pour le jeu $(A, -A)$. Montrez que de l'existence d'un équilibre on peut dériver le théorème minimax, à savoir :

$$max_x(min_y\ x^T Ay) = min_y(max_x\ x^T Ay)\ . \tag{3.22}$$

   Donc on peut dire que le théorème de Nash (existence d'un équilibre) implique le théoreme minimax, mais notez que la preuve du théorème de Nash s'applique à des jeux plus généraux, elle est considérablement plus compliquée et elle ne suggère pas une méthode efficace pour calculer un équilibre.

6. On dispose aussi d'algorithmes spécialisés pour le calcul d'un équilibre de Nash [LH64, PNS08].
7. Cette analyse est instructive mais pas essentielle pour la suite.

2. Supposons que l'équation (3.22) est valide et supposons aussi :

$$\begin{aligned} min_y\ \overline{x}^T Ay &= max_x(min_y\ x^T Ay), \\ max_x\ x^T A\overline{y} &= min_y(max_x\ x^T Ay). \end{aligned}$$

   Donc $\overline{x}$ est une meilleure stratégie possible pour *Max* (*Ligne*) et $\overline{y}$ est une meilleure stratégie possible pour *Min* (*Colonne*). Montrez que dans ce cas $(\overline{x}, \overline{y})$ est un équilibre de Nash pour le jeu $(A, -A)$.

Analyse

On s'intéresse maintenant aux jeux généraux (on ne suppose pas $B = -A$). On dénote par $e_i = (0, \cdots, 0, 1, 0, \cdots, 0)^T$ le i-ème vecteur de la base canonique. Soient :

M	$= max\{(e_i)^T Ay \mid i = 1, \ldots, m\}$	(gain maximum stratégie pure *Ligne*)
r_i	$= M - (e_i)^T Ay \quad i = 1, \ldots, m$	(regret stratégie pure i *Ligne*)
I_M	$= \{i \in \{1, \ldots, m\} \mid (e_i)^T Ay = M\}$	(coups avec gain maximum de *Ligne*)
S_x	$= \{i \in \{1, \ldots, m\} \mid x_i > 0\}$	(support stratégie de *Ligne*).

3. Montrez que x est une meilleure réponse à y ssi $S_x \subseteq I_M$ et $x^T Ay = M$. Donc en termes géométriques une stratégie mixte est une meilleure réponse ssi elle est une combinaison convexe de stratégies pures qui maximisent le gain de *Ligne*.
4. En adaptant le résultat précédent, donnez les conditions nécessaires et suffisantes pour que y soit une meilleure réponse à x.
5. Utilisez les deux points précédents pour donner une condition nécessaire et suffisante pour que (x, y) soit un équilibre.
6. Par définition, le regret r_i est une quantité non-négative. En fonction de A, proposez une borne supérieure pour r_i.

Modélisation et test

En utilisant l'analyse du problème ci-dessus, on cherche à modéliser le calcul d'un équilibre de Nash comme un problème d'optimisation linéaire mixte (avec des variables continues et des variables discrètes). L'idée de départ est d'introduire $m+n$ variables discrètes qui varient sur $\{0,1\}$ et qui décrivent le support des stratégies de x et de y. On rappelle que d'après l'analyse on sait que si $x_i > 0$ alors $r_i = 0$ et si $x_i = 0$ alors r_i peut être borné supérieurement.

seule question à faire

7. Programmez une fonction qui prend un argument un jeu (A, B) et deux stratégies mixtes (x, y) et vérifie si les stratégies forment un équilibre de Nash pour le jeu.
8. Proposez et mettez en oeuvre une méthode pour transformer un jeu (A, B) en un problème d'optimisation linéaire mixte P avec la propriété que P a toujours une solution optimale et une solution optimale de P correspond à un équilibre du jeu. La méthode sera implémentée en python3 et PULP est la seule bibliothèque autorisée. Notez que plusieurs solutions sont possibles en fonction du choix des contraintes et de la fonction objectif. Par exemple, on peut faire en sorte que toute solution admissible de P soit un équilibre ou alors on peut admettre des solutions admissibles mais *pas* optimales qui ne correspondent pas à un équilibre.
9. Proposez un ou plusieurs jeu de tests pour votre modèle. Par exemple, un objectif pourrait être de tester le modèle sur des classes de *jeux intéressantes*. La bibliothèque

GAMUT[8] en propose un certain nombre. Un autre objectif pourrait être de tester le modèle sur des problèmes dont la difficulté est notoire. On pourra consulter [CS08], pour une simple réduction polynomiale du problème SAT au problème de trouver un équilibre de Nash qui a certaines propriétés.[9]

8. `http://gamut.stanford.edu/`
9. Un équilibre de Nash existe toujours donc pour avoir une réduction il faut mettre des contraintes sur l'équilibre qu'on cherche. Dans [DGP09], on montre que le problème de calculer un équilibre de Nash est complet pour une classe de complexité un peu exotique nommée PPAD pour *Polynomial Parity Arguments on Directed Graphs*.

3.8.8 TP Logistique du dernier kilomètre

Votre entreprise souhaite développer un nouveau logiciel d'aide à la décision dans le domaine de la 'logistique du dernier kilomètre'.

Un premier problème

Comme étape préliminaire on vous demande d'évaluer différents algorithmes disponibles dans un contexte simplifié. On suppose qu'il faut servir n clients qui sont distribués de façon uniforme sur une grille carrée de taille $n^2 \times n^2$ (deux clients peuvent se trouver à la même position de la grille). La distance entre deux clients qui se trouvent aux coordonnées (i, j) et (k, ℓ) est $|k-i|+|\ell-j|$ (aussi connue comme distance de Manhattan). Le centre de distribution se trouve (environ) au centre du carré et le problème est de trouver un circuit qui part du centre de distribution passe par chaque client exactement une fois et revient au centre de distribution en couvrant une distance qui est aussi petite que possible. Il est donc question ici du problème classique du commis voyageur avec une distance métrique (abrégé dans la suite en Δ-TSP). Les algorithmes seront implémentés en python3 et PULP est la seule bibliothèque autorisée. Les approches qu'on vous propose d'analyser et qu'on décrit de façon sommaire dans la suite sont les suivantes :

- programmation dynamique,
- optimisation linéaire en nombres entiers,
- recherche locale,
- approximation.

Adapter ces approches aux problèmes de logistique dont il est question fait partie de votre *travail de modélisation*. Pour chaque approche vous allez :

- déterminer de façon expérimentale la taille de problèmes que vous pouvez traiter en moins d'une minute avec un ordinateur premier prix,
- pour les méthodes qui peuvent retourner une solution non-optimale, analyser de façon expérimentale la qualité de la solution obtenue.

Vous aurez sans doute besoin de :

- une fonction qui prend en entrée le paramètre n et génère un problème de façon aléatoire,
- une fonction qui vérifie que le circuit retourné comme solution est bien une solution (pas forcement optimale) du problème,
- une fonction qui calcule la différence entre la solution trouvée et la solution optimale (ou une borne inférieure/supérieure de la solution optimale si la taille du problème ne permet pas de calculer une solution optimale).

Dans la suite, on dénote par $N = \{1, \ldots, n\}$ l'ensemble des noeuds d'un graphe complet et par 1 un noeud fixé dans N. Si $i, j \in N$ alors on dénote par $d(i, j)$ leur distance.

Programmation dynamique

Soit $X \subseteq N \backslash \{1\}$. Pour $j \in N \backslash X$ on dénote par $t(X, j)$ la longueur d'un parcours de longueur minimale qui démarre du noeud 1, passe exactement une fois par chaque élément de X et termine au noeud j. On remarque que :

$$t(\emptyset, j) = \begin{cases} 0 & \text{si } 1 = j \\ d(1, j) & \text{autrement.} \end{cases}$$

Par ailleurs, si $X \neq \emptyset$ alors on a la récurrence :

$$t(X, j) = min\{t(X\backslash\{i\}, i) + d(i, j) \mid i \in X\} \ .$$

La solution optimale de Δ-TSP est donc $t(N\backslash\{1\}, 1)$. On note qu'il est facile d'adapter les équations de façon à calculer un circuit optimal en plus du coût minimum. Par ailleurs, les techniques standards de la programmation dynamique permettent d'éviter de recalculer plusieurs fois la fonction t sur les mêmes arguments. Il va sans dire que toute estimation de l'efficacité de cette approche suppose une mise-en-oeuvre d'une de ces techniques.

Optimisation linéaire en nombres entiers

On formule le problème Δ-TSP comme un problème d'optimisation linéaire sur les entiers et on demande à un solveur (dans notre cas PULP) de trouver une solution en appliquant un mélange savant de *branch and cut*. On sait que l'efficacité du solveur peut dépendre fortement de la formulation du problème ; on en présente une largement documentée dans la littérature.

On commence par introduire $n \cdot (n-1)$ variables entières $x_{i,j}$ pour $i, j \in N$ et $i \neq j$ avec l'interprétation suivante : $x_{i,j} = 1$ si le noeud j suit immédiatement le noeud i dans le circuit et $x_{i,j} = 0$ autrement. Avec cette interprétation, la quantité qu'on souhaite minimiser est :

$$\Sigma_{i,j \in N, i \neq j} \ d(i, j) \cdot x_{i,j} \ . \tag{3.18}$$

Dans un circuit on doit avoir la propriété que chaque noeud $i \in N$ a exactement un successeur et un prédécesseur immédiat. On exprime cette propriété avec les $2 \cdot n$ équations suivantes :

$$\begin{array}{ll} \Sigma_{j \in N, i \neq j} \ x_{i,j} = 1 & i = 1, \ldots, n \\ \Sigma_{i \in N, i \neq j} \ x_{i,j} = 1 & j = 1, \ldots, n \ . \end{array} \tag{3.19}$$

On remarque que les équations plus les $n \cdot (n-1)$ inégalités $x_{i,j} \geq 0$ impliquent que $0 \leq x_{i,j} \leq 1$. On remarque aussi que le problème de minimisation avec les contraintes en question peut être vu comme un problème de couplage pondéré dans un graphe biparti avec $2 \cdot n$ noeuds et on sait que le relâchement sur les rationnels de ce problème admet toujours une solution optimale entière (la matrice des contraintes est tum).

Malheureusement les contraintes (3.19) ne suffisent pas pour exprimer le fait que l'on souhaite calculer un *seul* circuit qui passe par tous les noeuds. En effet, il est parfaitement possible de satisfaire les contraintes (3.19) en utilisant plusieurs circuits disjoints qui recouvrent tous les noeuds dans N. En général, la "solution" qu'on obtient est une borne inférieure de la solution optimale du problème Δ-TSP.

Si on veut exclure tout circuit avec 2 éléments qui ne passe pas par le noeud 1 on peut ajouter les contraintes :

$$\Sigma_{i \in N\backslash X, j \in X} \ x_{i,j} \geq 1 \quad \sharp X = 2, X \subseteq N\backslash\{1\}$$

Il y a un nombre quadratique en n de ces contraintes. Et si on veut exclure les circuits avec 3 éléments on a un nombre cubique en n de ces contraintes,... Si on veut exclure tout circuit qui ne passe pas par 1 on doit produire un nombre *exponentiel* de contraintes de la forme :

$$\Sigma_{i \in N\backslash X, j \in X} \ x_{i,j} \geq 1 \quad \sharp X \geq 2, X \subseteq N\backslash\{1\} \tag{3.20}$$

Clairement si n n'est pas petit, on doit envisager une méthode pour générer les contraintes à la demande (voir par exemple [PR91]). Ici on discute une autre façon d'exclure les circuits qui ne passent pas par 1 qui s'exprime avec un nombre quadratique en n de contraintes.[5]

On introduit des nouvelles variables u_i pour $i = 2, \ldots, n$ (notez qu'il n'y a pas de variable u_1 !) avec l'intuition que u_i exprime le "potentiel" du noeud i dans le circuit. Pour fixer les idées on peut identifier le potentiel u_i avec la position du noeud i dans le circuit mais on verra qu'on n'a pas besoin d'être aussi spécifique.

On introduit les $(n-1)\cdot(n-2)$ contraintes suivantes :

$$u_i - u_j + x_{i,j} \cdot (n-1) \leq (n-2) \qquad 2 \leq i, j \leq n, i \neq j \tag{3.21}$$

D'abord on va voir que ces contraintes interdisent tout circuit qui ne passe pas par le noeud 1. En effet soit $i_1, \ldots, i_k, i_1$ un tel circuit. Alors $x_{i_1,i_2} = \cdots = x_{i_{k-1},i_k} = x_{i_k,i_1} = 1$ et les contraintes (3.21) se simplifient en :

$$\begin{array}{lcl} u_{i_1} - u_{i_2} + (n-1) & \leq & (n-2) \\ \cdots & \leq & \cdots \\ u_{i_{k-1}} - u_{i_k} + (n-1) & \leq & (n-2) \\ u_{i_k} - u_{i_1} + (n-1) & \leq & (n-2)\ . \end{array}$$

Si on additionne ces contraintes on dérive $k(n-1) \leq k(n-2)$, $k \geq 2$ qui n'a pas de solution. Donc tout circuit doit passer par le noeud 1 et par la contrainte (3.19) le noeud 1 a exactement un prédécesseur et un successeur immédiat. Il y a donc un seul circuit qui passe par tous les noeuds.

Il faut maintenant vérifier que les contraintes (3.21) n'excluent pas des solutions admissibles. Considérons un circuit qui passe par tous les noeuds $1, i_2, \ldots, i_n, 1$. On peut satisfaire les contraintes (3.21) en posant $u_{i_2} = 1, \ldots, u_{i_n} = (n-1)$. En effet, si $x_{i,j} = 0$ la contrainte devient :

$$u_i - u_j \leq (n-2)$$

qui est satisfaite car $u_i, u_j \in \{1, \ldots, n-1\}$. Et si $x_{i,j} = 1$ alors la contrainte devient $u_i - u_j + (n-1) \leq (n-2)$ soit :

$$u_j \geq u_i + 1\ .$$

On note que dans la modélisation il faut juste imposer que les variables $x_{i,j}$ sont des variables entières non-négatives alors que les variables u_i peuvent varier librement sur les rationnels (les contraintes $1 \leq u_i \leq (n-1)$ ainsi que les contraintes u_i entier ne sont pas nécessaires).

Recherche locale

Soit $\gamma = x_1, x_2, \ldots, x_n, x_1$ un circuit qui part de x_1 visite chaque noeud exactement une fois et retourne à x_1. Un circuit γ' est dans le voisinage $N(\gamma)$ de γ si γ' a la forme suivante pour $1 \leq i < i+2 \leq j \leq n$:

$$x_1, x_2, \ldots, x_i, x_j, x_{j-1}, \ldots, x_{i+1}, x_{j+1}, \ldots, x_n, x_1$$

5. L'expérience suggère que cette méthode n'est pas très adaptée pour des problèmes de grande taille (voir, par exemple, [CCZ14] [page 63]) mais elle est plus simple à mettre en oeuvre que celle basée sur les contraintes (3.20).

Par exemple, si $\gamma = 1,2,3,4,5,1$, $i = 2$ et $j = 4$ alors $\gamma' = 1,2,4,3,5,1$. Si $d(\gamma) = \Sigma_{i=1,\ldots,n-1}\ d(i,i+1) + d(n,1)$ est le coût du circuit de départ, le coût de γ' est :

$$d(\gamma') = d(\gamma) - d(i,i+1) - d(j,j+1) + d(i,j) + d(i+1,j+1)\ .$$

On peut vérifier que :

- $N(\gamma)$ contient $n(n-3)/2$ éléments,
- les circuits dans $N(\gamma)$ sont exactement les circuits qu'on peut obtenir de γ en supprimant deux arêtes qui ne partagent pas de noeuds et ensuite en ajoutant deux nouvelles arêtes de façon à reconstituer un nouveau circuit,
- le choix des deux arêtes à supprimer détermine l'opération à faire pour reconstituer le circuit.

En ayant défini une notion de voisinage, l'algorithme de recherche locale peut être formulé très simplement :

1. on génère un circuit γ (par exemple, de façon aléatoire),
2. tant qu'il existe un $\gamma' \in N(\gamma)$ tel que $d(\gamma') < d(\gamma)$ on remplace γ par γ',
3. on retourne le circuit γ.

On note que si la convergence de l'algorithme est trop lente on peut utiliser un *time-out* et si elle est rapide on peut répéter l'expérience avec un autre circuit initial. Dans les deux cas, le circuit retourné par l'algorithme n'est pas forcement optimal.

Approximation

On cherche à avoir une garantie sur la qualité de la solution obtenue. Pour ce faire, on applique un algorithme classique (Prim, Kruskal,...) pour obtenir un arbre de recouvrement minimum T. On note que le poids de T est une borne inférieure au coût d'un circuit optimal pour Δ-TSP (pourquoi ?). On considère ensuite un parcours préfixe de l'arbre T (on visite les noeuds et ensuite on visite récursivement les fils). Si on ajoute à la fin de ce parcours la racine de l'arbre on obtient un circuit pour Δ-TSP et on vérifie que le coût du parcours est au plus 2 fois le coût de T. Il en suit que le coût du circuit calculé par cette méthode est au plus deux fois le coût du circuit optimal. Il va sans dire qu'il s'agit d'une borne pour le pire des cas ; en pratique la solution trouvée peut être plus proche de la solution optimale. Il y a une autre méthode connue comme *algorithme de Christofides* (voir, par exemple, [PS82]) qui permet de calculer en temps polynomial un circuit dont le coût est au plus $1,5$ fois le coût du circuit optimal ; sa mise en oeuvre est un peu plus compliquée et optionnelle.

Le problème "réel"

Le problème "réel" que le logiciel devra traiter est un peu plus compliqué que celui décrit ci-dessus. On sait que :

- le nombre de clients est de l'ordre de 10^3,
- le centre de distribution dispose d'environ $k \approx 10$ livreurs qui peuvent opérer en parallèle,
- le but est de trouver une stratégie de distribution qui permet aux k livreurs de terminer le travail de distribution le plus tôt possible (donc chaque client est traité exactement par un livreur et le travail termine quand le dernier livreur a terminé son circuit).

On aimerait savoir quelle approche parmi celles évoquées est la plus adaptée au problème "réel". Vous allez choisir l'approche qui vous semble la plus prometteuse parmi les quatre, l'adapter au problème réel et rédiger un rapport qui décrit les résultats de votre expérience.

(page 71)

4.4.2 TP Un vecteur le plus proche du réseau et réduction de bases

Soit $B = \{b_1, \ldots, b_n\}$ une base et v un vecteur. On s'intéresse au problème de déterminer un vecteur dans $\Lambda(B)$ qui est parmi les plus proches à v au sens de la distance euclidienne. On peut reformuler ce problème comme un problème de décision et montrer qu'il est complet pour la classe NP [vEB81].

1. On suppose que $v = \Sigma_{i=1,\ldots,n} x_i b_i$ avec $x_i \in \mathbf{R}$ pour $i = 1, \ldots, n$. Montrez que si B est *orthogonale* alors :
$$\Sigma_{i=1,\ldots,n} \lceil x_i \rfloor b_i \in \Lambda(B) \tag{4.11}$$
est parmi les vecteurs de $\Lambda(B)$ les plus proches à v.
2. Si la base n'est pas orthogonale on peut quand même calculer les coefficients x_i et ensuite les arrondir comme dans (4.11) pour obtenir un vecteur du réseau $\Lambda(B)$. On se propose d'évaluer de façon expérimentale la qualité de la solution en fonction de l'orthogonalité de la base.

(a) Programmez une fonction qui prend en paramètre la dimension n et un nombre naturel positif d et génère une base B en choisissant les coefficients avec probabilité uniforme dans l'intervalle $[-d, d]$ (il s'agit d'une heuristique pour générer une base assez proche d'une base orthogonale).

(b) Programmez une fonction qui prend en paramètre la dimension n et deux nombres naturels positifs a et b et génère une matrice unimodulaire U en composant a transformations unimodulaires élémentaires choisies de façon aléatoire parmi : permuter deux colonnes, changer le signe d'une colonne, additionner à une colonne k fois une autre colonne où k est un entier dans l'intervalle $[-b, b]$.

(c) Programmez une fonction qui prend en paramètre la dimension n et un nombre naturel positif b et génère un vecteur v de dimension n à coefficients entiers dans l'intervalle $[-b, b]$.

(d) Programmez la méthode LLL pour réduire une base.

(e) Appliquez les fonctions des points précédents pour générer une base B, une transformation unimodulaire U et un vecteur v et approchez la distance d'un vecteur le plus proche à v pour les 4 bases suivantes qui génèrent le même réseau : B, B après réduction, BU et BU après réduction. Quelle conclusion tirez-vous de cette expérience ?[3]